

SmartHydro

Scaletta operativa di implementazione

Ordine consigliato, logica di sviluppo, formule e test minimi

Progetto	Distributed System for Automated Hydroponic Greenhouses
Gruppo	Edoardo Menegotto, Veronica Pizzigati, Chiara Serafini
Versione	2.0, scaletta in ordine di implementazione
Data	06/07/2026

Questo documento non sostituisce RASD, Design Document e codice finale. Serve come piano operativo per capire in quale ordine costruire il progetto e quale risultato deve essere verificato a ogni passo.

Indice operativo

1. 1. Idea guida e criterio di sviluppo incrementale
2. 2. Stack tecnologico e struttura della repository
3. 3. Modello logico e matematico da tenere fisso
4. 4. Scaletta di implementazione in ordine consigliato
5. 5. Contratti dati e API minime
6. 6. Piano di test e criteri di accettazione
7. 7. Divisione del lavoro nel gruppo
8. 8. Checklist finale prima della consegna

1. Idea guida e criterio di sviluppo incrementale

Il progetto deve essere implementato come una serie di versioni funzionanti sempre più complete. La prima versione non deve avere subito dashboard, backend completo e tre strategie di controllo. Deve invece dimostrare che una zona di serra simulata evolve nel tempo, legge sensori simulati, calcola comandi e aggiorna attuatori. Solo dopo si aggiungono ricette, più zone, guasti, backend e interfaccia web.

La regola pratica è questa: ogni fase deve produrre un programma eseguibile o un modulo testabile, non solo codice parziale. In questo modo è più facile fare debug, documentare il progetto e arrivare alla discussione finale con una storia tecnica chiara.

Ordine consigliato: prima dominio e simulazione, poi controllo locale, poi guasti, poi comunicazione, poi backend, poi dashboard, infine integrazione e test end to end.

2. Stack tecnologico e struttura della repository

Parte	Linguaggio	Scopo	Tool consigliati
Edge Controller	C++17 o C++20	Simulazione della serra, sensori, attuatori, loop di controllo, FSM, strategie, fault recovery	CMake, VS Code, Catch2 o GoogleTest, nlohmann/json, Doxygen
Cloud Backend	Python 3.11+	Gestione ricette, zone, eventi, telemetria, configurazioni, API per dashboard	FastAPI, Uvicorn, SQLite, SQLAlchemy oppure SQLAlchem, pytest
Dashboard	HTML, CSS, JavaScript	Interfaccia agronomo per ricette, stato zone, strategia scelta, guasti simulati, grafici	Vanilla JS, Chart.js, fetch API, eventualmente Bootstrap
Documentazione	Markdown, PlantUML o draw.io	RASD, Design Document, diagrammi UML, README, manuale demo	GitHub, draw.io, PlantUML, Doxygen

Struttura consigliata della repository

```
SmarthHydro/  
  docs/  
    RASD/  
    DesignDocument/  
    diagrams/  
  edge_controller/  
    include/  
    src/  
    tests/  
    CMakeLists.txt  
  backend/  
    app/  
    tests/  
    requirements.txt  
  dashboard/  
    index.html  
    styles.css  
    app.js  
  demo/
```

recipes/
scenarios/
README.md

La struttura separa chiaramente documenti, edge, backend e dashboard. È utile perché il corso valuta coerenza tra requisiti, design e implementazione, quindi ogni modifica dovrebbe lasciare traccia sia nei documenti sia nel codice.

3. Modello logico e matematico da tenere fisso

Prima di programmare le classi conviene fissare il modello minimo del sistema. Questo evita di cambiare continuamente nomi e responsabilità durante lo sviluppo.

3.1 Variabili principali

Simbolo o nome	Significato	Tipo
z	Identificativo della zona o serra indipendente	intero o stringa
k	Indice discreto del tempo di simulazione	intero
dt	Passo temporale della simulazione, per esempio 1 secondo o 1 minuto simulato	double
x_z(k)	Stato fisico reale della zona z	[temperatura, umidità, pH]
y_z(k)	Misura letta dai sensori, eventualmente disturbata o guasta	[temperatura, umidità, pH]
r_z(k)	Setpoint richiesto dalla ricetta nella fase corrente	[temperatura, umidità, pH]
u_z(k)	Comando agli attuatori	[heater, fan, humidifier, acidPump, basePump]

3.2 Aggiornamento della serra simulata

Per non complicare troppo il progetto, il modello fisico può essere discreto e semplificato. Serve solo a generare dati credibili e a mostrare che i controllori producono un effetto osservabile.

```
T(k+1) = T(k) + dt*(aH*heater - aF*fan + bT*(Text - T(k)) + noiseT)
H(k+1) = H(k) + dt*(aU*humidifier - aD*fan + bH*(Hext - H(k)) + noiseH)
pH(k+1) = pH(k) + dt*(aB*basePump - aA*acidPump + noisepH)
```

Ogni comando deve essere saturato: $0 \leq u_i(k) \leq 1$.

I coefficienti aH, aF, aU, aD, aB, aA non devono essere realistici al 100 per cento. Devono solo rendere visibile che un attuttore aumenta o diminuisce una variabile. Il rumore può essere piccolo, per esempio compreso tra meno 0.05 e più 0.05.

3.3 Errore di controllo e saturazione

```
e_v(k) = r_v(k) - y_v(k)
clamp(value, min, max) = min(max(value, min), max)
```

Dove v appartiene a {temperatura, umidità, pH}.

Tutte le strategie devono usare la stessa interfaccia: ricevono misure, setpoint e stato precedente, restituiscono comandi agli attuatori. Questo permette di applicare il pattern Strategy senza cambiare il resto del codice.

4. Scaletta di implementazione in ordine consigliato

Le fasi sono pensate come commit successivi. Dopo ogni fase il progetto deve compilare o avviarsi, e deve esistere almeno un test o una demo minima.

Fase 0. Preparazione repository e contratti minimi

Obiettivo	Avere un progetto vuoto ma ordinato, con compilazione C++, ambiente Python e cartelle documentali già pronte.
-----------	---

Cosa implementare	Creare repository privata, cartelle principali, file README, CMakeLists, requirements Python, cartella demo. Aggiungere un file JSON di esempio per una ricetta.
Come implementarlo	Partire da un progetto piccolo: un main C++ che stampa versione, un backend FastAPI con endpoint health, una pagina HTML statica.
Logica da rispettare	Il contratto iniziale e che ogni modulo abbia un punto di ingresso chiaro: edge executable, backend app, dashboard index.
Output atteso	Repository avviabile con tre comandi: build edge, run backend, open dashboard.
Test minimo	Verificare che CMake compili, che FastAPI risponda a /health, che README descriva come avviare.
Tool consigliati	GitHub, VS Code, CMake, Python venv

Fase 1. Classi di dominio comuni

Obiettivo	Definire i dati base che saranno usati da tutti i moduli.
Cosa implementare	Implementare ZoneId, EnvironmentValues, ActuatorCommand, Recipe, RecipePhase, TelemetrySample, Event, FaultType, StrategyType.
Come implementarlo	In C++ creare struct semplici e funzioni di serializzazione JSON. In Python creare modelli Pydantic equivalenti.
Logica da rispettare	La stessa informazione deve avere gli stessi nomi logici in edge, backend e dashboard. Questo riduce gli errori di integrazione.
Output atteso	File model.hpp in C++, models.py in Python, recipe_example.json nella cartella demo.
Test minimo	Testare lettura di una ricetta JSON e controllo di validità dei campi obbligatori.
Tool consigliati	nlohmann/json, Pydantic

Fase 2. Ricette e selezione della fase colturale

Obiettivo	Calcolare automaticamente quale fase della coltura è attiva e quali setpoint devono essere usati.
Cosa implementare	Implementare RecipeScheduler con startTime, durata delle fasi e funzione getCurrentSetpoints(zone, time).
Come implementarlo	Ogni ricetta contiene fasi ordinate: Germination, Vegetative, Flowering or Fruiting, Harvest. Ogni fase ha durata, target e tolleranze.
Logica da rispettare	Se elapsedTime cade nell'intervallo cumulativo della fase i , allora i setpoint attivi sono quelli della fase i . Formula: $\text{phase}(k) = i$ se $\sum(d_{0\dots d_{i-1}}) \leq \text{elapsed}(k) < \sum(d_{0\dots d_i})$.
Output atteso	Una funzione che, dato il tempo simulato, restituisce target temperatura, umidità e pH.
Test minimo	Provare una ricetta con tre fasi e verificare che al cambio tempo cambino anche i setpoint.
Tool consigliati	Unit test C++

Fase 3. Simulatore fisico della zona

Obiettivo	Creare una serra simulata che evolve nel tempo anche senza backend e senza dashboard.
Cosa implementare	Implementare GreenhouseZoneSimulator con stato reale x , ambiente esterno, rumore, metodo step(command, dt).
Come implementarlo	Usare formule discrete semplici per temperatura, umidità e pH. I comandi devono essere compresi tra 0 e 1. Il simulatore non deve conoscere le strategie di controllo.
Logica da rispettare	Dato $x(k)$ e $u(k)$, il simulatore produce $x(k+1)$. La separazione tra simulatore e controller serve a distinguere impianto controllato e logica di controllo.
Output atteso	Un programma demo che applica comandi fissi e mostra che T , H e pH cambiano nel verso atteso.
Test minimo	Testare heater alto, la temperatura sale. Fan alto, temperatura e umidità scendono. BasePump alto, pH sale.
Tool consigliati	Catch2 o GoogleTest

Fase 4. Adapter per sensori e attuatori simulati

Obiettivo	Separare la logica di controllo dai dispositivi, anche se i dispositivi sono simulati.
Cosa implementare	Definire interfacce ISensor e IActuator. Implementare TemperatureSensorAdapter, HumiditySensorAdapter, PHSensorAdapter e ActuatorAdapter.
Come implementarlo	Il controller deve usare solo le interfacce, non direttamente il simulatore. Il sensore legge dallo stato reale e restituisce misura y. L attuatore riceve comando e lo passa al simulatore.
Logica da rispettare	Misura nominale: $y_v(k) = x_v(k) + noise_v(k)$. In caso di fault la misura viene alterata secondo il tipo di guasto.
Output atteso	Sensori e attuatori simulati sostituibili in futuro con dispositivi reali.
Test minimo	Testare che un sensore nominale restituisca valori vicini allo stato reale e che un attuatore modifichi il comando applicato.
Tool consigliati	C++ interface, Adapter pattern

Fase 5. Loop principale dell Edge Controller

Obiettivo	Avere un controllo locale funzionante per una sola zona, senza backend.
Cosa implementare	Implementare EdgeController con ciclo: leggi sensori, scegli setpoint, calcola comando, applica attuatori, aggiorna simulatore, produce telemetria.
Come implementarlo	Il loop deve essere deterministico e semplice. Per la demo si può usare un ciclo for di N passi invece di un vero sistema real time.
Logica da rispettare	Ordine logico: $y(k) = readSensors()$, $r(k) = scheduler.getSetpoints()$, $u(k) = strategy.compute(r,y)$, $apply(u)$, $simulator.step(u,dt)$, $publish(sample)$.
Output atteso	Demo locale con una zona che segue una ricetta semplice.
Test minimo	Eseguire 100 o 500 passi e verificare che i valori si avvicinino ai target senza uscire dai limiti fisici.
Tool consigliati	C++, log CSV

Fase 6. Prima strategia di controllo, soglie

Obiettivo	Implementare la strategia più semplice e robusta, utile anche come fallback in modalità degradata.
Cosa implementare	Creare interfaccia IControlStrategy e classe ThresholdControlStrategy.
Come implementarlo	Per ogni variabile confrontare misura e target con una tolleranza. Sotto al limite inferiore si attiva l attuatore che aumenta la variabile. Sopra al limite superiore si attiva quello che la riduce.
Logica da rispettare	Se $y_v < r_v - tol_v$ allora comando $increase_v = 1$. Se $y_v > r_v + tol_v$ allora comando $decrease_v = 1$. Altrimenti entrambi restano 0.
Output atteso	Controller a soglie selezionabile e funzionante nella demo.
Test minimo	Testare tre casi: sotto soglia, dentro soglia, sopra soglia.
Tool consigliati	Strategy pattern

Fase 7. Seconda strategia di controllo, PID semplificato

Obiettivo	Aggiungere una strategia più continua e parametrizzabile.
Cosa implementare	Implementare PIDControlStrategy con parametri Kp, Ki, Kd per temperatura, umidità e pH. Salvare errore precedente e integrale.
Come implementarlo	Il PID restituisce intensità tra 0 e 1. Se il comando PID è positivo si usa l attuatore che aumenta la variabile, se negativo si usa quello che la riduce.
Logica da rispettare	$e(k) = r(k) - y(k)$, $I(k) = I(k-1) + e(k) * dt$, $D(k) = (e(k) - e(k-1)) / dt$, $u = clamp(Kp * e + Ki * I + Kd * D, -1, 1)$.
Output atteso	Seconda strategia intercambiabile senza modificare EdgeController.
Test minimo	Testare che un errore positivo generi comando positivo e che la saturazione limiti u in [-1,1].

Tool consigliati	Unit test C++
------------------	---------------

Fase 8. Terza strategia, controllo predittivo semplificato

Obiettivo	Aggiungere una strategia che sembri diversa dal PID ma resti implementabile.
Cosa implementare	Implementare PredictiveControlStrategy con orizzonte H piccolo, per esempio 3 passi, e pochi comandi candidati predefiniti.
Come implementarlo	Per ogni comando candidato si simula una previsione semplificata e si sceglie quello con costo minore. Non serve un vero MPC complesso.
Logica da rispettare	Costo: $J = \sum_v w_v (r_v - \hat{x}_v)^2 + \lambda \sum_i u_i^2$. Si sceglie il comando con J minimo.
Output atteso	Terza strategia selezionabile a runtime.
Test minimo	Testare che, se temperatura è bassa, il comando con heater alto abbia costo inferiore rispetto a heater spento.
Tool consigliati	Strategy pattern, simulatore predittivo interno semplificato

Fase 9. Cambio strategia a runtime

Obiettivo	Permettere all'agronomo o al backend di cambiare politica di controllo senza riavviare il sistema.
Cosa implementare	Implementare StrategyManager o metodo EdgeController::setStrategy(type).
Come implementarlo	Mantenere EdgeController dipendente solo dall'interfaccia IControlStrategy. Quando arriva una richiesta, sostituire l'oggetto strategia attivo.
Logica da rispettare	Il comportamento cambia perché cambia l'oggetto strategia, non perché EdgeController contiene if complessi su ogni algoritmo.
Output atteso	Demo locale: primi 100 passi a soglie, poi PID, poi predittivo.
Test minimo	Verificare che il cambio strategia produca eventi e non resettì la zona o la ricetta.
Tool consigliati	C++ polymorphism

Fase 10. FSM della coltura

Obiettivo	Rappresentare formalmente il ciclo di vita della coltura.
Cosa implementare	Creare CropLifecycleState con stati Germination, Vegetative, FloweringFruiting, Harvest. Implementare transizioni automatiche per tempo o manuali.
Come implementarlo	Il RecipeScheduler può usare questa FSM invece di una semplice somma delle durate. Ogni transizione genera un evento StateChanged.
Logica da rispettare	Stato iniziale Germination. Transizione alla fase successiva se $\text{elapsedInPhase} \geq \text{phaseDuration}$ oppure se arriva manualAdvance. Harvest è stato finale operativo.
Output atteso	FSM colturale documentabile con state diagram UML.
Test minimo	Testare sequenza completa di fasi e blocco oltre Harvest.
Tool consigliati	State pattern oppure enum più classe StateMachine

Fase 11. FSM operativa e recovery dai guasti

Obiettivo	Gestire stati di sistema diversi dal normale funzionamento.
Cosa implementare	Creare OperationalStateMachine con stati Nominal, Degraded, EmergencyLockdown, Maintenance.
Come implementarlo	In Nominal il controller usa la strategia scelta. In Degraded usa una strategia sicura a soglie o ignora la variabile con sensore guasto. In Emergency spegne attuatori rischiosi e porta la zona in stato sicuro.
Logica da rispettare	Transizioni: Nominal -> Degraded su fault recoverable. Degraded -> Nominal se recovered e stabile per R tick. Degraded -> Emergency se fault critical o valore fuori limite. Emergency -> Maintenance su intervento umano. Maintenance -> Nominal solo con reset manuale e sensori validi.
Output atteso	Gestione chiara di sicurezza e recovery automatico.

Test minimo	Testare ogni transizione con eventi sintetici.
Tool consigliati	State pattern, unit test

Fase 12. Fault simulation su sensori e attuatori

Obiettivo	Simulare guasti credibili e far reagire il sistema.
Cosa implementare	Implementare FaultInjector con fault: sensor_stuck, sensor_offset, sensor_dropout, actuator_stuck_off, actuator_stuck_on, actuator_slow_response.
Come implementarlo	Il fault injector modifica il comportamento di Adapter o Simulator. Deve essere attivabile da scenario JSON o API backend.
Logica da rispettare	Sensor stuck: $y(k)=\text{costante}$. Offset: $y(k)=x(k)+\text{bias}$. Dropout: misura assente. Actuator stuck off: comando applicato = 0 anche se richiesto. Stuck on: comando applicato = 1.
Output atteso	Una demo in cui il sistema entra in Degraded o Emergency a seguito di un guasto.
Test minimo	Testare ogni guasto e verificare lo stato operativo atteso.
Tool consigliati	Scenario JSON, unit test

Fase 13. Fault detection e condizioni logiche di sicurezza

Obiettivo	Non limitarsi a simulare il guasto, ma riconoscerlo attraverso regole.
Cosa implementare	Implementare FaultDetector con regole su range fisico, salto massimo, assenza misura, mancata risposta attuatore.
Come implementarlo	Il detector riceve ultimi campioni e ultimi comandi. Produce FaultEvent con gravita e zona coinvolta.
Logica da rispettare	Regole: misura invalida se y fuori [minPhys, maxPhys], se y e missing per N tick, se $ y(k)-y(k-1) > \text{maxRate} \cdot dt$. Attuatore sospetto se comando alto per M tick ma $ x(k)-x(k-M) < \text{deltaMin}$.
Output atteso	Rilevamento automatico dei guasti principali.
Test minimo	Testare dati nominali senza falsi allarmi e casi guasti con allarme.
Tool consigliati	Unit test, log eventi

Fase 14. Observer per telemetria ed eventi

Obiettivo	Rendere il sistema reattivo e prepararlo alla comunicazione con backend e dashboard.
Cosa implementare	Implementare Subject/EventBus nell'edge. Observer possibili: CsvLogger, ConsoleLogger, BackendClient.
Come implementarlo	EdgeController non deve conoscere direttamente chi riceve i dati. Pubblica eventi e campioni; gli observer li consumano.
Logica da rispettare	Eventi minimi: TelemetrySample, StateChanged, FaultDetected, StrategyChanged, RecipePhaseChanged, EmergencyTriggered.
Output atteso	Log locale CSV e console già funzionanti prima di collegare il backend.
Test minimo	Testare che ogni step produca telemetry e che un guasto produca FaultDetected.
Tool consigliati	Observer pattern

Fase 15. Backend Python con API minime

Obiettivo	Aggiungere il server centrale senza appesantire subito la dashboard.
Cosa implementare	Implementare FastAPI con storage SQLite per ricette, zone, telemetria ed eventi. Esporre endpoint REST.
Come implementarlo	Il backend deve essere inizialmente semplice: riceve telemetry dall'edge, salva eventi, restituisce ricette e comandi di configurazione.
Logica da rispettare	Il backend non controlla direttamente la fisica. Coordina configurazioni e rende visibili dati e comandi.
Output atteso	Server FastAPI con documentazione automatica Swagger.
Test minimo	Test API con pytest e chiamate curl o HTTPie.
Tool consigliati	FastAPI, SQLite, SQLAlchemy, pytest

Fase 16. Client backend dentro Edge Controller

Obiettivo	Collegare edge e backend mantenendo il sistema funzionante anche offline.
Cosa implementare	Implementare BackendClient in C++: GET ricette, POST telemetria, GET comandi pendenti, POST eventi.
Come implementarlo	Se backend non disponibile, edge continua in modalita locale e salva log. Questo migliora robustezza e rende la demo meno fragile.
Logica da rispettare	Politica offline: se request fallisce, retry dopo R tick e publish evento BackendUnavailable. Il controllo locale non si ferma.
Output atteso	Telemetria visibile nel backend mentre il simulatore gira.
Test minimo	Avviare backend, edge e verificare che i campioni compaiano nel database.
Tool consigliati	cpp-http lib o libcurl, nlohmann/json

Fase 17. Multi zona e multi ricetta

Obiettivo	Estendere da una zona a piu zone indipendenti.
Cosa implementare	Creare GreenhouseManager che contiene piu ZoneController. Ogni zona ha ricetta, strategia, stato fisico e FSM propria.
Come implementarlo	Il manager esegue un tick su tutte le zone. In una prima versione puo farlo in sequenza, senza thread. Il parallelismo reale non e necessario per dimostrare la gestione concorrente.
Logica da rispettare	Per ogni z in zones: zone[z].tick(dt). Le zone non devono condividere stato fisico. Possono condividere backend client e logger.
Output atteso	Demo con almeno due zone, per esempio basilico e pomodoro, con target diversi.
Test minimo	Verificare che cambiare strategia o guasto su una zona non modifichi l'altra.
Tool consigliati	C++ containers, integration test

Fase 18. Dashboard agronomo

Obiettivo	Permettere all'utente finale di vedere e modificare il sistema.
Cosa implementare	Creare pagine o sezioni: overview zone, dettaglio zona, ricette, cambio strategia, fault injection, eventi.
Come implementarlo	Usare fetch per chiamare backend. Per i grafici usare Chart.js. Aggiornare dati con polling ogni 1 o 2 secondi, sufficiente per la demo.
Logica da rispettare	La dashboard non deve contenere logica di controllo. Deve inviare comandi al backend e visualizzare stato, dati e notifiche.
Output atteso	Interfaccia web usabile per la demo finale.
Test minimo	Test manuale: creare ricetta, assegnarla a zona, cambiare strategia, inserire fault e vedere evento.
Tool consigliati	HTML, CSS, JavaScript, Chart.js

Fase 19. Scenario demo end to end

Obiettivo	Preparare una dimostrazione completa e ripetibile.
Cosa implementare	Creare script o istruzioni per avviare backend, dashboard ed edge. Preparare scenario JSON con due zone, due ricette, cambio strategia e guasto.
Come implementarlo	La demo deve raccontare una sequenza chiara: sistema nominale, controllo setpoint, cambio ricetta o fase, fault, recovery, dashboard aggiornata.
Logica da rispettare	Scenario minimo: t0 avvio due zone. t1 strategia PID su zona A. t2 fault sensore pH su zona A. t3 Degraded. t4 recovery o Emergency. t5 reset manuale.
Output atteso	Demo ripetibile senza dover cliccare troppe cose a mano.
Test minimo	Eseguire la demo da zero seguendo README e correggere ogni passaggio ambiguo.
Tool consigliati	README, script bash o Python

Fase 20. Testing, documentazione e pulizia finale

Obiettivo	Rendere il progetto credibile dal punto di vista Software Engineering.
Cosa implementare	Scrivere test unitari, test API, test integrazione, aggiornare UML, generare Doxygen, sistemare README e manuale demo.
Come implementarlo	I test devono dimostrare requisiti funzionali e robustezza. Non serve testare tutto, ma serve coprire parti decisive: controller, FSM, fault detection, API, scenario completo.
Logica da rispettare	Verifica interna: il codice implementa il design. Validazione esterna: la demo risponde allo scenario SmarthHydro.
Output atteso	Consegna coerente: documenti, codice, test e demo allineati.
Test minimo	Eseguire test automatici e scenario end to end prima del commit finale.
Tool consigliati	pytest, Catch2 o GoogleTest, Doxygen, draw.io

5. Contratti dati e API minime

Questi contratti vanno fissati presto, perché collegano edge, backend e dashboard. Possono essere semplificati, ma non conviene cambiarli continuamente.

5.1 JSON ricetta

```
{
  "recipeId": "tomato_basic",
  "cropName": "Tomato",
  "phases": [
    {
      "name": "Germination",
      "durationHours": 48,
      "temperatureTarget": 24.0,
      "humidityTarget": 75.0,
      "phTarget": 6.0,
      "temperatureTolerance": 1.0,
      "humidityTolerance": 5.0,
      "phTolerance": 0.2
    }
  ]
}
```

5.2 Telemetria inviata dall edge

```
{
  "zoneId": "zone_A",
  "timestamp": "2026-07-06T14:00:00",
  "strategy": "PID",
  "operationalState": "Nominal",
  "cropPhase": "Vegetative",
  "measured": {"temperature": 23.8, "humidity": 68.0, "ph": 6.1},
  "setpoint": {"temperature": 24.0, "humidity": 70.0, "ph": 6.0},
  "commands": {"heater": 0.2, "fan": 0.0, "humidifier": 0.4, "acidPump": 0.0, "basePump": 0.0}
}
```

5.3 Endpoint backend minimi

Metodo	Endpoint	Responsabilità
GET	/health	Verifica che il backend sia attivo
POST	/api/recipes	Crea o aggiorna una ricetta
GET	/api/recipes	Restituisce le ricette disponibili

POST	/api/zones	Crea una zona o assegna una ricetta
GET	/api/zones	Restituisce stato sintetico di tutte le zone
POST	/api/zones/{zoneId}/strategy	Richiede cambio strategia
POST	/api/zones/{zoneId}/faults	Richiede simulazione guasto
POST	/api/telemetry	Riceve campioni dall edge
GET	/api/events	Restituisce eventi e notifiche

6. Piano di test e criteri di accettazione

Il piano di test deve essere costruito in parallelo al codice. Il progetto non deve arrivare alla fine con una sola demo manuale.

Livello	Cosa testare	Esempi di casi
Unit test C++	Simulator, controller, FSM, fault detector	PID con errore positivo, soglie dentro tolleranza, transizione Nominal to Degraded
Unit test Python	Validazione modelli, API, database	Ricetta senza phases rifiutata, GET zones restituisce stato corrente
Integration test	Comunicazione edge backend	Edge invia telemetry, backend la salva, dashboard la legge
System test	Scenario completo	Due zone, due ricette, cambio strategia, guasto sensore, recovery
Robustness	Input errati e componenti non disponibili	Backend spento, misura mancante, fault critico, ricetta non valida

Criteri minimi di accettazione della demo

- Almeno due zone funzionano contemporaneamente con ricette diverse.
- La fase della coltura cambia nel tempo e modifica i setpoint.
- Si può cambiare strategia di controllo a runtime.
- Sono implementate almeno tre strategie: soglie, PID semplificato, predittivo semplificato.
- Un guasto simulato viene rilevato e produce Degraded o EmergencyLockdown.
- La dashboard mostra telemetria, stato operativo, fase colturale, strategia ed eventi.
- Il backend salva almeno ricette, stato zone, telemetria ed eventi.
- Il README permette di avviare la demo da zero.

7. Divisione del lavoro nel gruppo

Persona o ruolo	Responsabilità principale	Responsabilità secondaria
Responsabile Edge C++	Simulator, Adapter, EdgeController, strategie, FSM, fault detection	Doxygen e test C++
Responsabile Backend	FastAPI, database, API, validazione modelli, test Python	Contratti JSON e integrazione
Responsabile Dashboard e documenti	Interfaccia web, grafici, scenario demo, UML e documenti	Test end to end e README

La divisione non deve impedire integrazione continua. Ogni persona dovrebbe poter avviare tutto il progetto localmente, anche se lavora principalmente su un solo blocco.

8. Checklist finale prima della consegna

- RASD aggiornato con scenari, requisiti, domain model, FSM e assunzioni.
- Design Document aggiornato con component diagram, class diagram, sequence diagram, pattern e traceability.
- Codice coerente con i nomi usati nei documenti.
- README con comandi di avvio e scenario demo.
- Test eseguibili con un comando per edge e un comando per backend.
- Cartella demo con ricette e scenari già pronti.
- Nessun file temporaneo inutile nella repository.
- Presentazione orale costruita sulla sequenza: requisiti, architettura, implementazione, demo, test.

Riferimenti interni usati per impostare la scaletta

Project Proposal SmartHydro: scope, linguaggi, core features e pattern richiesti.

SmartHydro, scaletta operativa di implementazione

SE4A Project Guideline: deliverable richiesti, repository GitHub, RASD, Design Document e implementazione.

Materiale SE4A su modular design, UML, FSM, design pattern e V&V.